MADRAS INSTITUTE OF TECHNOLOGY

ANNA UNIVERSITY

CHENNAI - 600 044

DEPARTMENT OF ELECTRONICS ENGINEERING

Regulation 2023

JULY 2025 – NOVEMBER 2025

EC23020 - PIC MICROCONTROLLERS

ASSIGNMENT - 1

SUBMITTED BY

GIRISURYAA L S
2023504022
PRE-FINAL YEAR STUDENT

SUBMITTED TO

Dr. K. VEERAPAN
FACULTY OF DEPARTMENT OF
ELECTRONICS ENGINEERING

INDEX

1. Design and develop a Smart Street Lighting System using a PIC microcontroller and Embedded C. The system should automatically turn ON the streetlights at night and OFF during the day using an LDR (Light Dependent Resistor). Additionally, to save power, the lights should glow dim when no vehicle is detected and bright when a vehicle passes using an IR sensor.

AIM

To design and develop a Smart Street Lighting System in PIC16F877A microcontroller with Embedded C Programming.

COMPONENTS REQUIRED

Software

1. Proteus 8 Professional
2. MikroC PRO for PIC

Proteus Schematic Components

1. PIC16F877A
2. Power Supply
3. Crystal Oscillator
4. 22pF Capacitors
5. 10kΩ , 300Ω Resistors
6. Switch for Reset
7. LDR Sensor
8. IR Sensor
9. LED

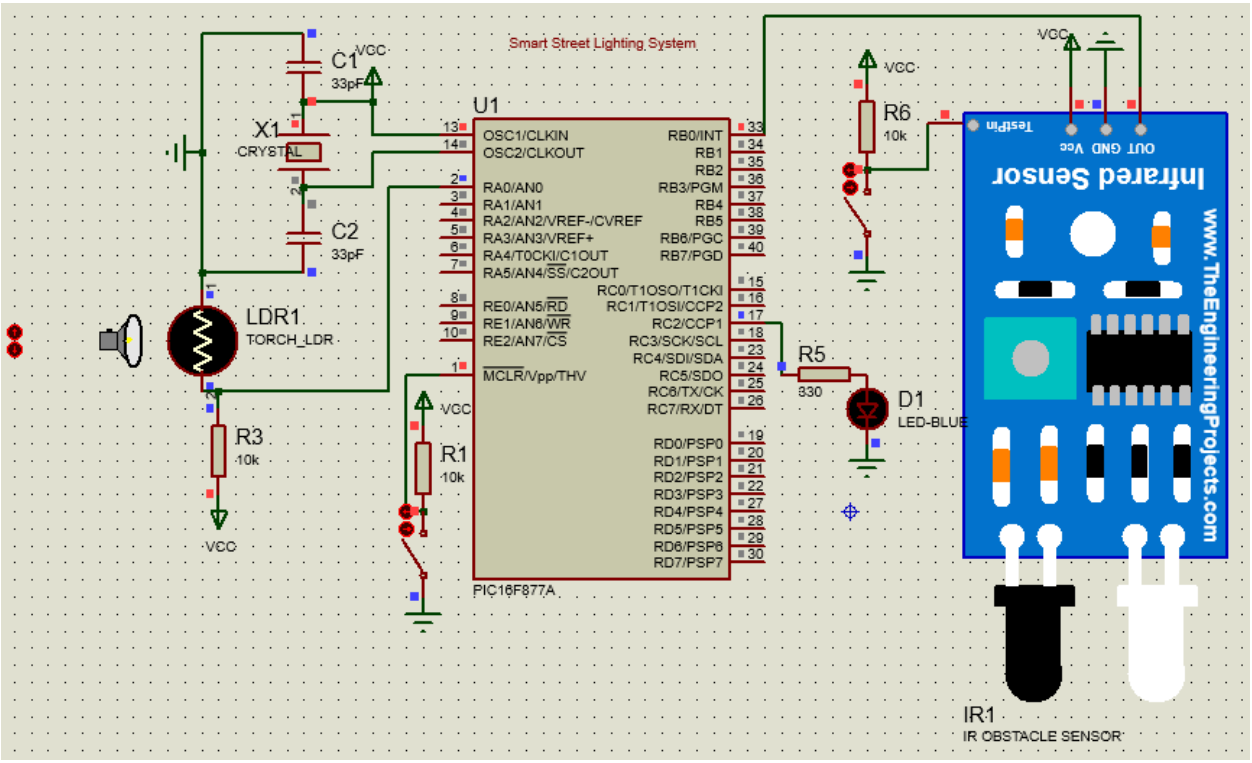
PROCEDURE

1. Create a new project in Proteus.
2. Place components (MCU, sensors, LEDs, etc.) and wire them.
3. Create a new project in mikroC and write the Embedded C Program.
4. Upload the program's HEX file to the microcontroller.
5. Set component properties .
6. Click "Start Simulation" .
7. Observe outputs (LEDs, motors, LCD, etc.) and note behavior.
8. Record data or take screenshots for the report.

EMBEDDED C CODE

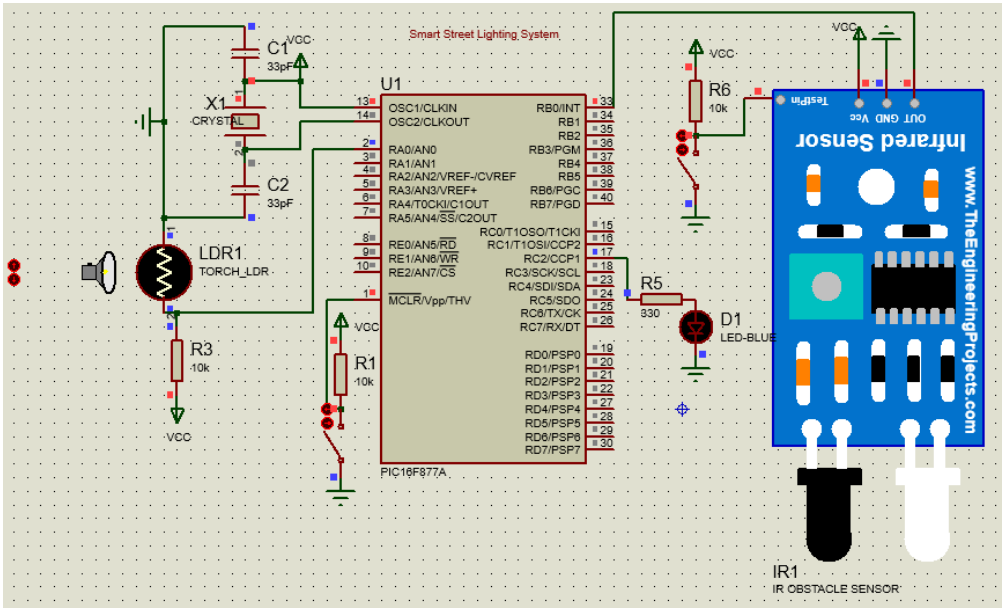
```
// Smart Street Light System
// MCU: PIC16F877A @ 20MHz
// Compiler: MikroC PRO for PIC

void main(){
  unsigned int LDR;  // Variable to store LDR sensor value
  unsigned int ir;       // Variable to store IR sensor value

  // Port Configuration
  TRISA.F0 = 1;    // Set RA0/AN0 as input (LDR sensor)
  TRISB.F0 = 1;    // Set RB0 as input (IR sensor)
  TRISC.F2 = 0;    // Set RC2/CCP1 as output (PWM to LED/Streetlight)

  ADCON1 = 0x80;   // Configure ADC: Right justified, Vref=Vdd/Vss, RA0 analog

  PWM1_Init(5000); // Initialize PWM module at 5 kHz frequency
  PWM1_Start();       // Start PWM on CCP1 (RC2)

  while(1){
    // Read sensor values
    LDR = ADC_Read(0);  // Read analog value from LDR on RA0/AN0
    ir = PORTB.F0;            // Read digital input from IR sensor at RB0

    // Check LDR value for day/night condition
    if(LDR >= 150) {          // Night time (LDR detects darkness)

      if(ir == 0)
        PWM1_Set_Duty(250); // Vehicle detected → Bright light (approx. 98% duty)
      else
        PWM1_Set_Duty(150); // No vehicle → Dim light (approx. 60% duty)
    }
    else {
      PWM1_Set_Duty(0);  // Day time → Light OFF
    }
  }
      }
```

CIRCUIT DIAGRAM

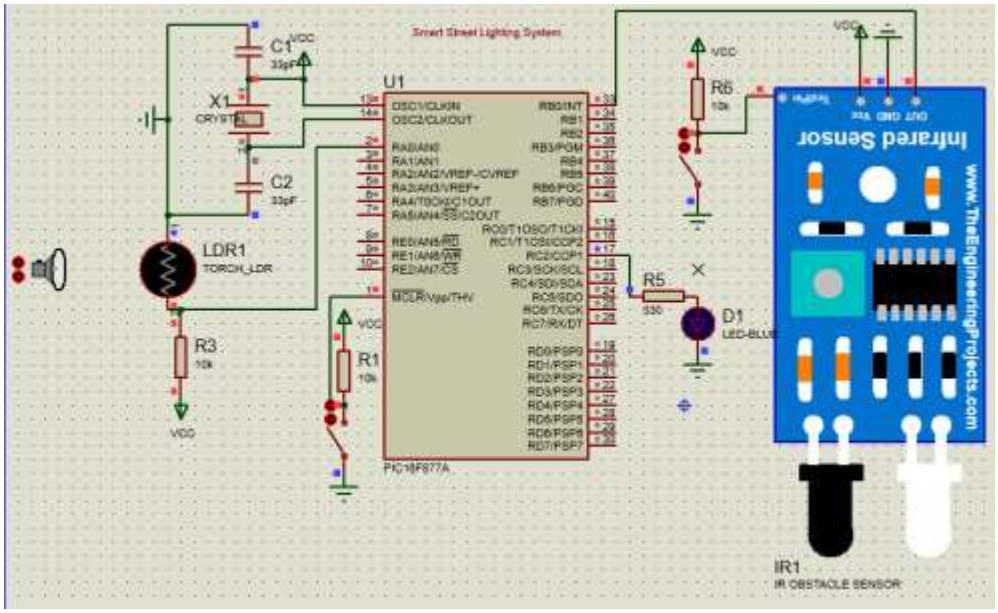


OUTPUT

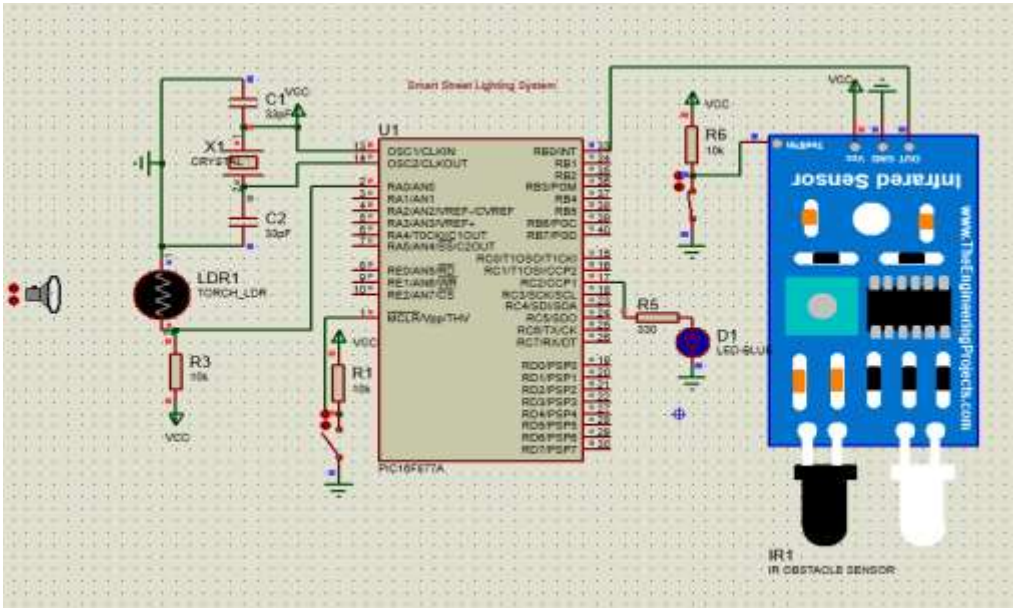
a. Day Time (LED does not glow)



b. Night Time without Vehicles (LED glows dim)



c. Night Time with Vehicles (LED glows bright)



RESULT

Thus the Smart Street Lightening System was successfully designed and developed in PIC16F877A microcontroller with Embedded C Programming and results are verified using simulation in Proteus.

2. Smart Temperature Control

Write a program in Embedded C for a PIC microcontroller that switches ON a fan if the room temperature exceeds 30°C (using LM35 sensor).

AIM

To design and develop a Smart Temperature Control in PIC16F877A microcontroller with Embedded C Programming.

COMPONENTS REQUIRED

Software

1. Proteus 8 Professional
2. MikroC PRO for PIC

Proteus Schematic Components

1. PIC16F877A
2. Power Supply
3. Crystal Oscillator
4. 22pF Capacitors
5. 10kΩ Resistors
6. Switch for Reset
7. LM35 Temperature Sensor
8. DC Motor

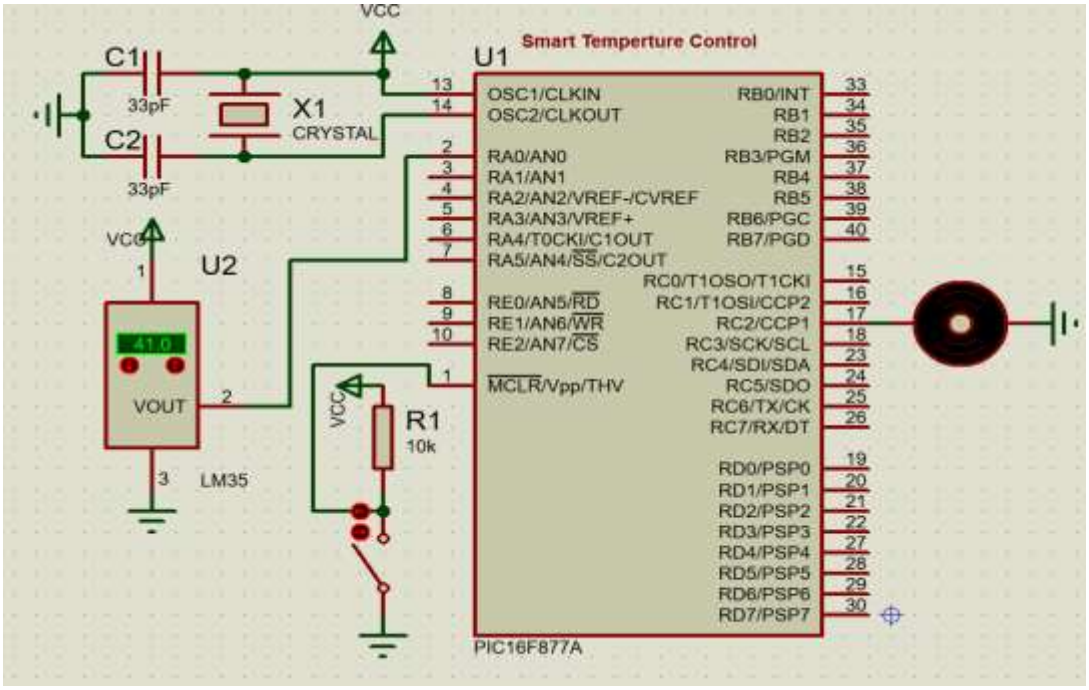
PROCEDURE

1. Create a new project in Proteus.
2. Place components (MCU, sensors, LEDs, etc.) and wire them.
3. Create a new project in mikroC and write the Embedded C Program.
4. Upload the program's HEX file to the microcontroller.
5. Set component properties .
6. Click "Start Simulation" .
7. Observe outputs (LEDs, motors, LCD, etc.) and note behavior.
8. Record data or take screenshots for the report.

EMBEDDED C CODE

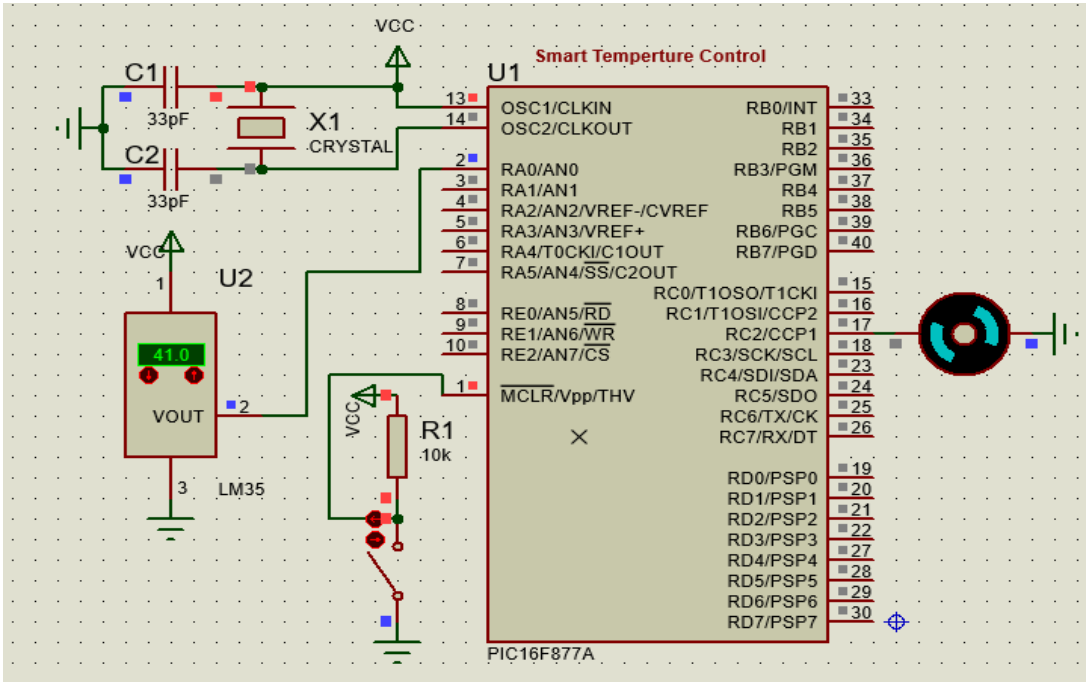
```
// Temperature Controlled Fan using LM35 + PWM
// MCU: PIC16F877A @ 20MHz
// Compiler: MikroC PRO for PIC

unsigned int adc_value;   // Stores ADC reading from LM35
float temperature;        // Stores calculated temperature (°C)

void main() {
                          // --- ADC & I/O Configuration ---
  ADCON1 = 0x80;      // Right justified, AN0 analog input enabled
  TRISA.F0 = 1;       // RA0/AN0 as input (LM35 temperature sensor)
  TRISC.F2 = 0;       // RC2/CCP1 as output (PWM to fan driver)

  // --- PWM Initialization ---
  PWM1_Init(5000);    // Initialize PWM at 5 kHz (suitable for motor control)
  PWM1_Start();       // Start PWM module
  PWM1_Set_Duty(0);   // Fan OFF initially

  while(1) {
    // --- Read temperature from LM35 ---
    adc_value = ADC_Read(0);              // Read ADC value from AN0
    temperature = adc_value * 4.88 / 10.0;    // Convert ADC to °C
    // Explanation: 1 ADC step = 4.88mV, LM35 = 10mV/°C

    // --- Fan Speed Control Logic ---
    if(temperature < 30) {
      PWM1_Set_Duty(0);     // Below 30°C → Fan OFF
    }
    else if(temperature >= 30 && temperature <= 35) {
      PWM1_Set_Duty(80);    // 30–35°C → Low speed
    }
    else if(temperature > 35 && temperature <= 40) {
      PWM1_Set_Duty(160);   // 36–40°C → Medium speed
    }
    else {
      PWM1_Set_Duty(255);   // Above 40°C → High speed
    }
  }
}
```

CIRCUIT DIAGRAM



OUTPUT

When Temperature is greater than 30 degree, then Fan will speed up the rpm based on Temperature



RESULT

Thus the Smart Temperature Control was successfully designed and developed in PIC16F877A microcontroller with Embedded C Programming and results are verified using simulation in Proteus.

3. Smart Energy Meter

How would you design a digital energy meter using PIC to measure and display current consumption on an LCD? Explain with Embedded C functions for ADC and LCD interface.

AIM

To design and develop a Smart Energy Meter in PIC16F877A microcontroller with Embedded C Programming.

COMPONENTS REQUIRED

Software

1. Proteus 8 Professional
2. MikroC PRO for PIC

Proteus Schematic Components

1. PIC16F877A
2. Power Supply
3. Crystal Oscillator
4. 22pF Capacitors
5. 10kΩ , 300Ω Resistors
6. Switch for Reset
7. LCD Display
8. IC ACS712ELCTR-30A-T
9. Ammeter to verify the output

PROCEDURE

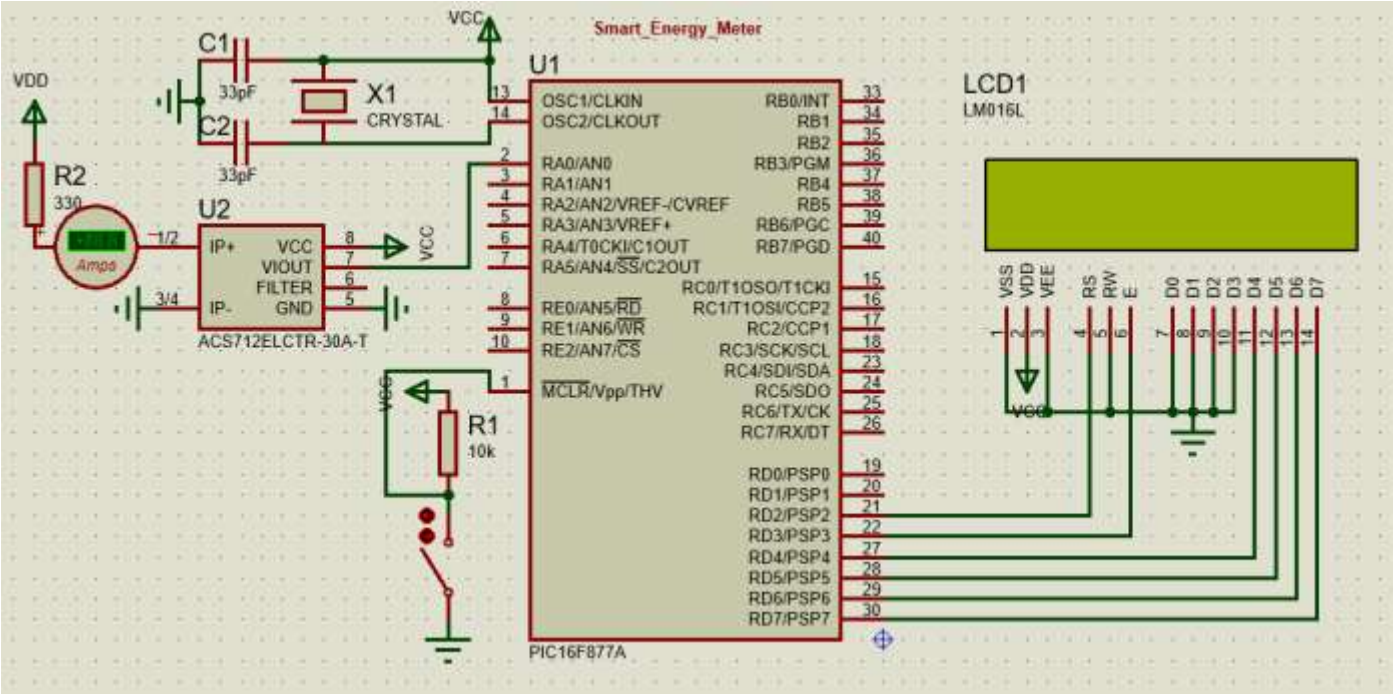
1. Create a new project in Proteus.
2. Place components (MCU, sensors, LEDs, etc.) and wire them.
3. Create a new project in mikroC and write the Embedded C Program.
4. Upload the program's HEX file to the microcontroller.
5. Set component properties .
6. Click "Start Simulation" .
7. Observe outputs (LEDs, motors, LCD, etc.) and note behavior.
8. Record data or take screenshots for the report.

EMBEDDED C CODE

```
// Energy Monitoring using ACS712 + LCD
// MCU: PIC16F877A @ 20MHz
// Compiler: MikroC PRO for PIC

// --- LCD Module Connections (4-bit mode) ---
sbit LCD_RS at RD2_bit;   // Register Select pin → RD2
sbit LCD_EN at RD3_bit;   // Enable pin → RD3
sbit LCD_D4 at RD4_bit;   // Data line D4 → RD4
sbit LCD_D5 at RD5_bit;   // Data line D5 → RD5
sbit LCD_D6 at RD6_bit;   // Data line D6 → RD6
sbit LCD_D7 at RD7_bit;   // Data line D7 → RD7

// --- LCD Direction Registers ---
sbit LCD_RS_Direction at TRISD2_bit;
sbit LCD_EN_Direction at TRISD3_bit;
sbit LCD_D4_Direction at TRISD4_bit;
sbit LCD_D5_Direction at TRISD5_bit;
sbit LCD_D6_Direction at TRISD6_bit;
sbit LCD_D7_Direction at TRISD7_bit;

// --- Main Program ---
void main(){
  char txt[16];                 // Buffer for LCD text
  unsigned int adc_val;         // ADC raw value
  float current, power, energy = 0;
  float voltage = 230.0;        // Assume constant mains voltage
  float sensitivity = 0.185;  // ACS712-05B sensitivity = 185 mV/A

  // Port configuration
  TRISA.F0 = 1;                 // AN0 (RA0) as analog input for ACS712
  TRISD = 0x00;                 // PORTD as output (LCD data/control)
  ADCON1 = 0x80;                // Configure RA0 as analog, right-justified result

  // LCD initialization
  Lcd_Init();
  Lcd_Cmd(_LCD_CLEAR);              // Clear display
  Lcd_Cmd(_LCD_CURSOR_OFF);   // Turn off cursor

  while(1){
    // --- Current sensing ---
    adc_val = ADC_Read(0);                  // Read ADC value from AN0
    current = (adc_val * 5.0 / 1023.0 - 2.5) / sensitivity;
    if(current < 0) current = -current;         // Remove negative offset error
```

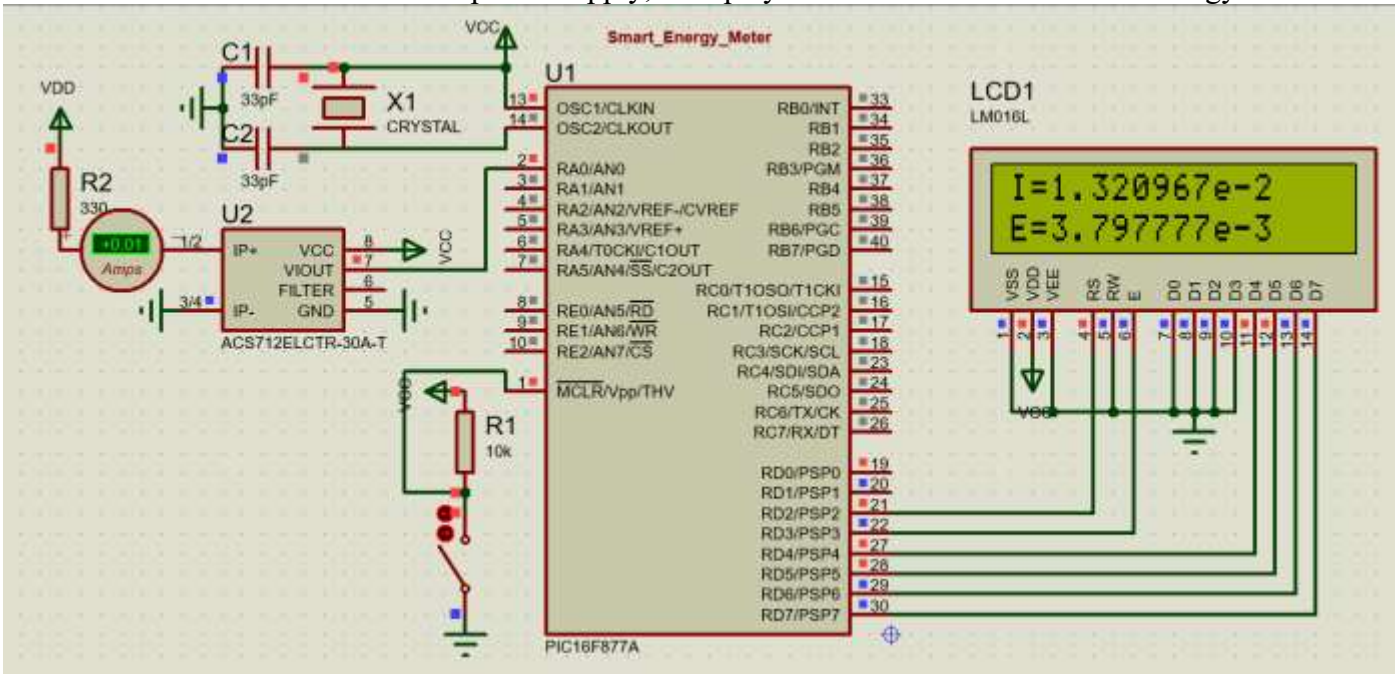
```
// --- Power & Energy calculation ---
 power = voltage * current;                  // Power = V × I (Watts)
 energy += power * 0.1 / 3600.0;             // Energy in kWh, 100 ms sampling

 // --- Display Current on LCD ---
 Lcd_Out(1,1,"I=");
 FloatToStr(current, txt); Lcd_Out(1,3,txt);  // Print current value

 // --- Display Energy on LCD ---
 Lcd_Out(2,1,"E=");
 FloatToStr(energy, txt); Lcd_Out(2,3,txt);   // Print energy value

 Delay_ms(100);                               // Sampling every 100 ms
}
  }
```

CIRCUIT DIAGRAM



OUTPUT

When the circuit is connected to the power supply, it displays the amount of Current and Energy



RESULT

Thus the Smart Energy Meter was successfully designed and developed in PIC16F877A microcontroller with Embedded C Programming and results are verified using simulation in Proteus.

4. Smart Health Monitoring System

How can you use a PIC microcontroller to measure a patient's heartbeat using a pulse sensor and display the BPM value on LCD? Provide program logic.

AIM

To design and develop a Smart Health Monitoring System in PIC16F877A microcontroller with Embedded C Programming.

COMPONENTS REQUIRED

Software

1. Proteus 8 Professional
2. MikroC PRO for PIC

Proteus Schematic Components

1. PIC16F877A
2. Power Supply
3. Crystal Oscillator
4. 22pF Capacitors
5. 10kΩ Resistors
6. Switch for Reset
7. Heart Beat Sensor
8. LCD Display

PROCEDURE

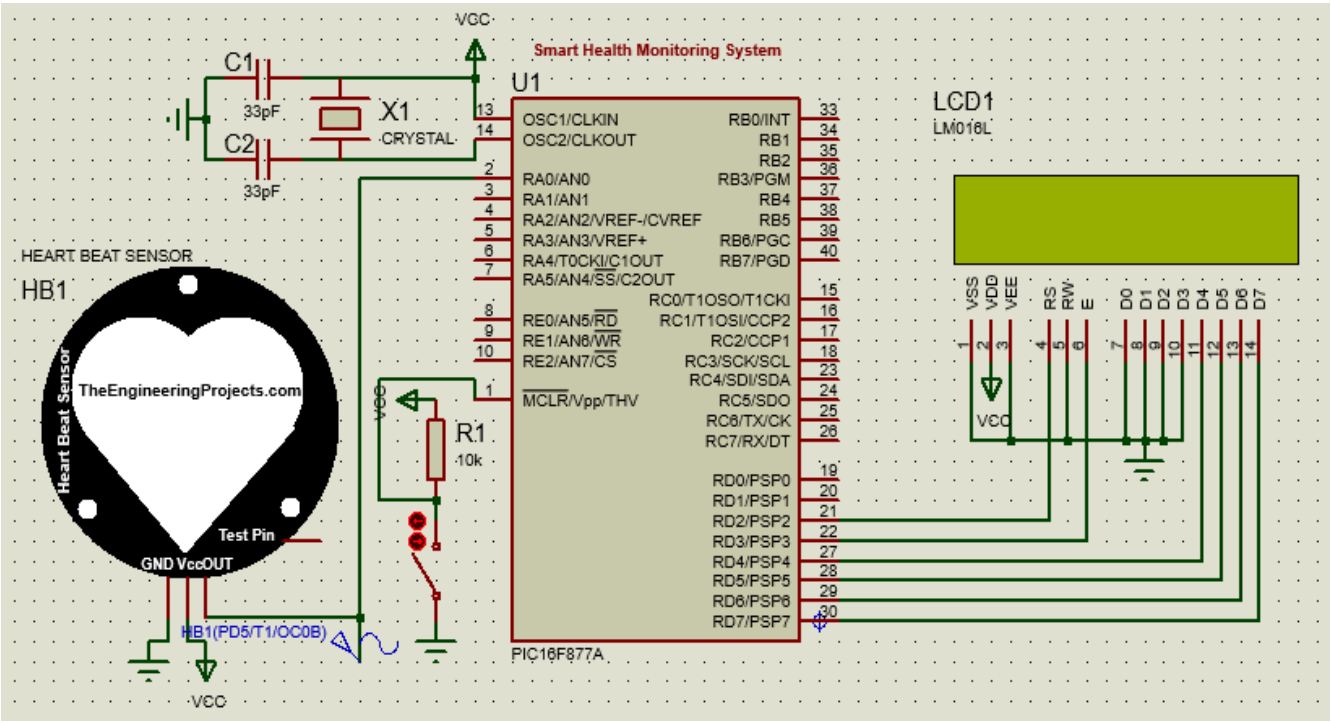
1. Create a new project in Proteus.
2. Place components (MCU, sensors, LEDs, etc.) and wire them.
3. Create a new project in mikroC and write the Embedded C Program.
4. Upload the program's HEX file to the microcontroller.
5. Set component properties .
6. Click "Start Simulation" .
7. Observe outputs (LEDs, motors, LCD, etc.) and note behavior.
8. Record data or take screenshots for the report.

EMBEDDED C CODE

```
// Heartbeat Monitor using Pulse Sensor + LCD
// MCU: PIC16F877A @ 20MHz
// Compiler: MikroC PRO for PIC

// --- LCD Module Connections ---
sbit LCD_RS at RD2_bit;   // LCD RS pin → RD2
sbit LCD_EN at RD3_bit;   // LCD EN pin → RD3
sbit LCD_D4 at RD4_bit;   // LCD D4 pin → RD4
sbit LCD_D5 at RD5_bit;   // LCD D5 pin → RD5
sbit LCD_D6 at RD6_bit;   // LCD D6 pin → RD6
sbit LCD_D7 at RD7_bit;   // LCD D7 pin → RD7

// --- LCD Direction Registers ---
sbit LCD_RS_Direction at TRISD2_bit;
sbit LCD_EN_Direction at TRISD3_bit;
sbit LCD_D4_Direction at TRISD4_bit;
sbit LCD_D5_Direction at TRISD5_bit;
sbit LCD_D6_Direction at TRISD6_bit;
sbit LCD_D7_Direction at TRISD7_bit;

void main() {
  unsigned int adc_val;      // Stores raw ADC value from pulse sensor
  int pulseCount = 0;        // Number of pulses counted in 5 sec
  int bpm = 0;               // Beats per minute
  int i;                     // Loop counter
  char txt[16];              // Buffer for LCD text
  int threshold = 512;       // Threshold for detecting pulse (mid-level ADC)
  int pulseDetected = 0;     // Flag to avoid multiple counts per beat

  // --- Port & ADC Configuration ---
  TRISA.F0 = 1;              // RA0/AN0 as input (pulse sensor)
  TRISD = 0x00;              // PORTD as output (LCD)
  ADCON1 = 0x80;             // Configure AN0 analog, others digital

  // --- LCD Initialization ---
  Lcd_Init();
  Lcd_Cmd(_LCD_CLEAR);           // Clear LCD
  Lcd_Cmd(_LCD_CURSOR_OFF);  // Hide cursor

  while(1){
    pulseCount = 0;                  // Reset counter for each cycle
```

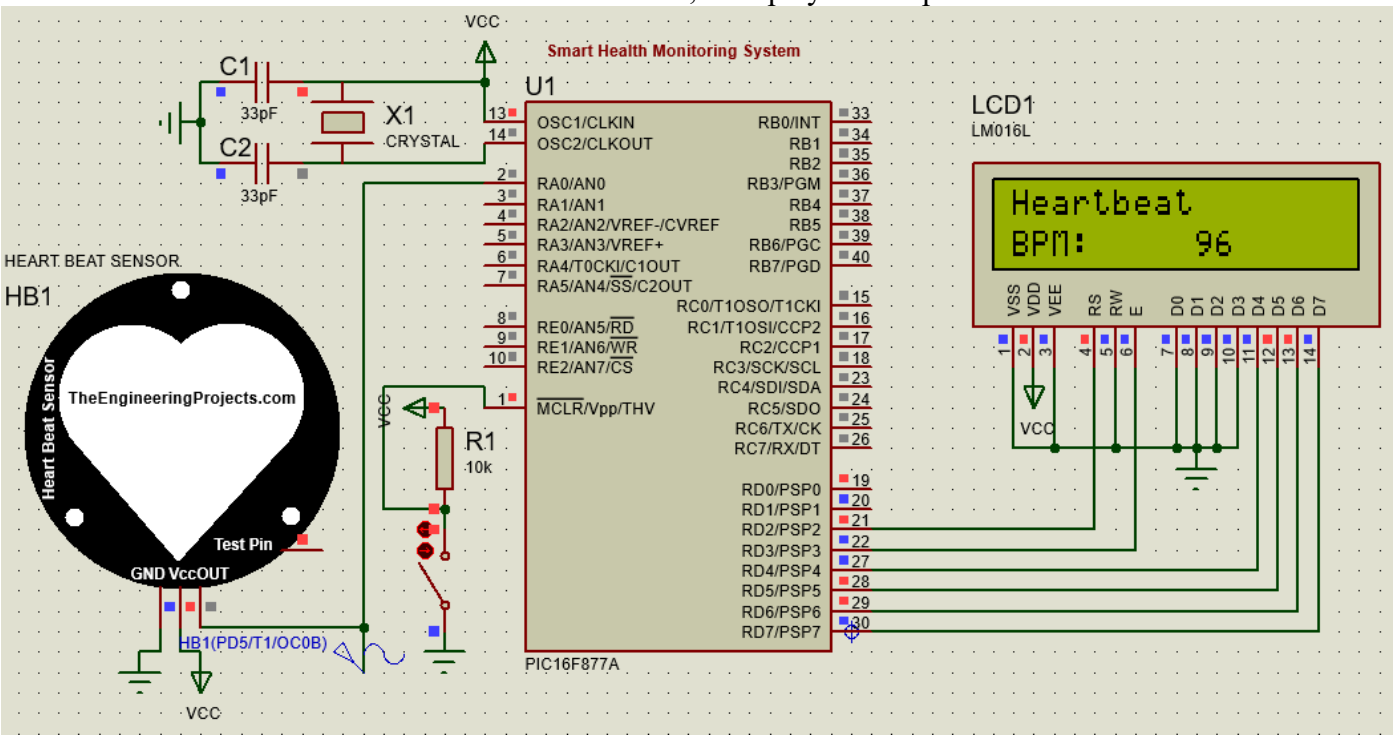
```
// --- Count pulses for 5 seconds ---
   for(i=0; i<50; i++){              // 50 samples × 100ms = 5 seconds
     adc_val = ADC_Read(0);    // Read analog value from AN0

     if(adc_val >= threshold && pulseDetected == 0){
       pulseCount++;                 // Beat detected → increase counter
       pulseDetected = 1;            // Lock until signal goes low
     }
     if(adc_val < threshold){
       pulseDetected = 0;            // Reset flag (ready for next beat)
     }
     delay_ms(100);                  // Sampling interval (100 ms)
   }
   // --- Convert to BPM (beats per minute) ---
   bpm = pulseCount * 12;          // (5 sec count × 12 = 60 sec)

   // --- Display on LCD ---
   Lcd_Cmd(_LCD_CLEAR);
   Lcd_Out(1,1,"Heartbeat");       // Line 1 title
   IntToStr(bpm, txt);             // Convert bpm integer to string
   Lcd_Out(2,1,"BPM: ");      // Line 2 label
   Lcd_Out(2,6,txt);              // Show BPM value
 }
   }
```

CIRCUIT DIAGRAM



OUTPUT

When the circuit is connected to the Heartbeat Sensor, it displays Beats per minute



RESULT

Thus the Smart Health Monitoring System was successfully designed and developed in PIC16F877A microcontroller with Embedded C Programming and results are verified using simulation in Proteus.

5. Smart Waste Management System

Write a PIC microcontroller Embedded C program for an ultrasonic sensor–based dustbin that opens automatically when someone approaches within 20 cm.

AIM

To design and develop a Smart Waste Management System in PIC16F877A microcontroller with Embedded C Programming.

COMPONENTS REQUIRED

Software

1. Proteus 8 Professional
2. MikroC PRO for PIC

Proteus Schematic Components

1. PIC16F877A
2. Power Supply
3. Crystal Oscillator
4. 22pF Capacitors
5. 10kΩ Resistors
6. Switch for Reset
7. Ultrasonic Sensor
8. Servomotor

PROCEDURE

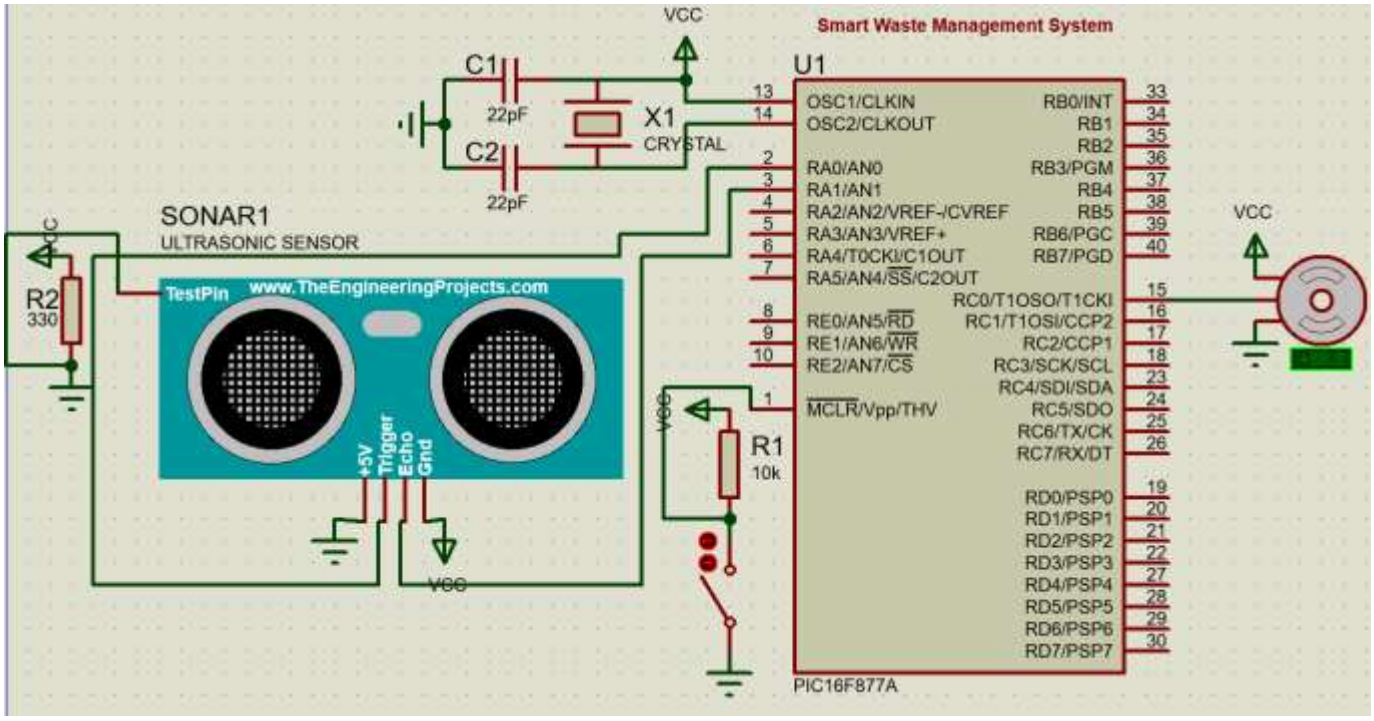
1. Create a new project in Proteus.
2. Place components (MCU, sensors, LEDs, etc.) and wire them.
3. Create a new project in mikroC and write the Embedded C Program.
4. Upload the program's HEX file to the microcontroller.
5. Set component properties .
6. Click "Start Simulation" .
7. Observe outputs (LEDs, motors, LCD, etc.) and note behavior.
8. Record data or take screenshots for the report.

EMBEDDED C CODE

```
/*
 * MCU: PIC16F877A @ 20MHz
 * Sensor: HC-SR04 (TRIG/ECHO)
 * Actuator: Servo Motor on RC0
 */

unsigned int distance;
unsigned int timeCount;
unsigned int i;

// === Servo Functions ===
void Servo_Open() {
   for(i = 0; i < 20; i++) {
      PORTC.F0 = 1;
      Delay_us(1500);             // ~90° position
      PORTC.F0 = 0;
      Delay_us(20000 - 1500);  // complete 20ms cycle
   }
}

void Servo_Close() {
   for(i = 0; i < 20; i++) {
      PORTC.F0 = 1;
      Delay_us(600);               // ~0° position
      PORTC.F0 = 0;
      Delay_us(20000 - 600);   // complete 20ms cycle
   }
}

// === Ultrasonic Distance Measurement ===
unsigned int getDistance() {
   // Send TRIG pulse
   PORTA.F0 = 1;
   Delay_us(10);
   PORTA.F0 = 0;

   // Wait for ECHO high
   while(!PORTA.F1);

   // Start Timer1
   TMR1H = 0;
   TMR1L = 0;
   T1CON.F0 = 1;
```

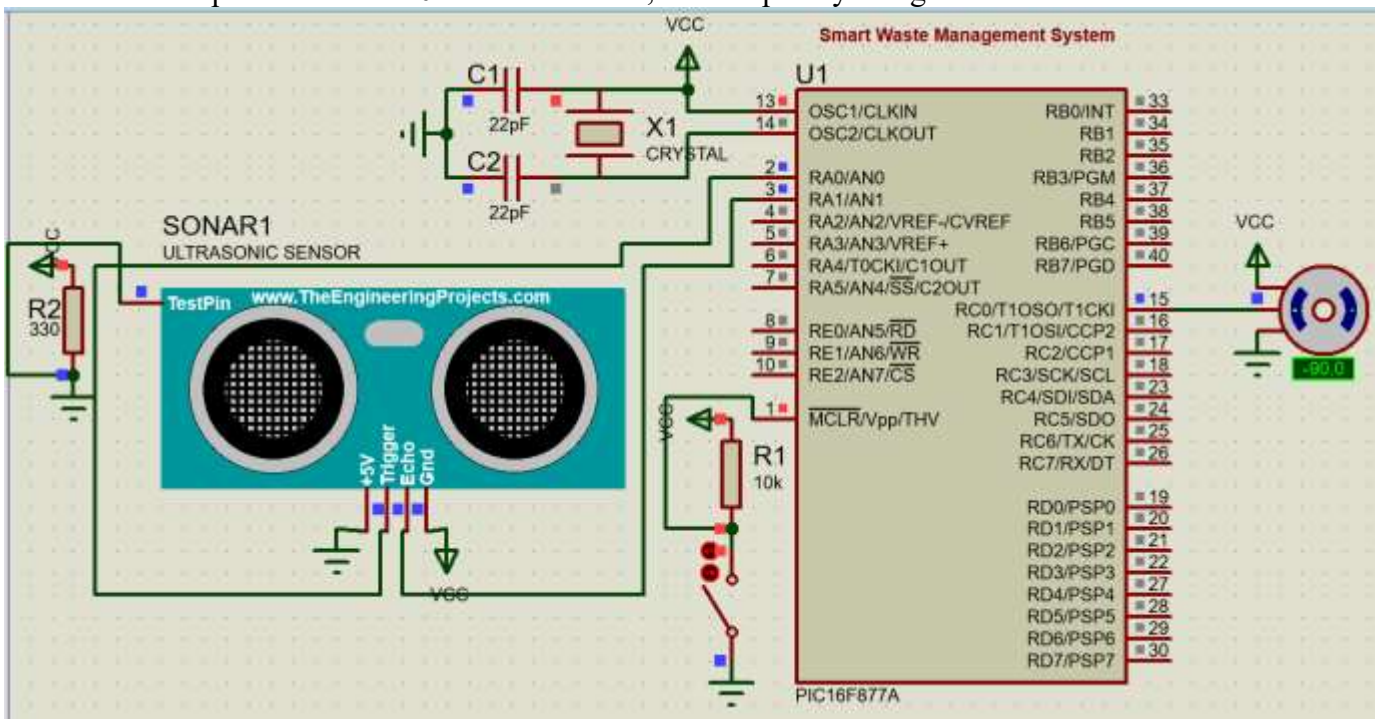
```
    // Wait for ECHO low
    while(PORTA.F1);

    // Stop Timer1
    T1CON.F0 = 0;

    // Read timer value
    timeCount = (TMR1H << 8) | TMR1L;

    // Convert to distance (cm)
    return (timeCount / 58);
}

// === MAIN ===
void main() {
    TRISA.F0 = 0;  // TRIG output
    TRISA.F1 = 1;  // ECHO input
    TRISC.F0 = 0;  // Servo output
    PORTA = 0;
    PORTC = 0;

    // Timer1 setup
    T1CON = 0x10;  // Prescaler 1:2, Timer1 OFF initially

    Delay_ms(100);
    Servo_Close();    // Start with closed lid

    while(1) {
        distance = getDistance();

        if(distance > 0 && distance <= 20) {
            Servo_Open();         // Open lid
            Delay_ms(5000);      // Wait 5 seconds
            Servo_Close();        // Close lid
        }

        Delay_ms(200);
    }
     }
```

CIRCUIT DIAGRAM



OUTPUT

When there is a person around 20cm near dustbin , it will open by using Servomotor



RESULT

Thus the Smart Waste Management System was successfully designed and developed in PIC16F877A microcontroller with Embedded C Programming and results are verified using simulation in Proteus.